

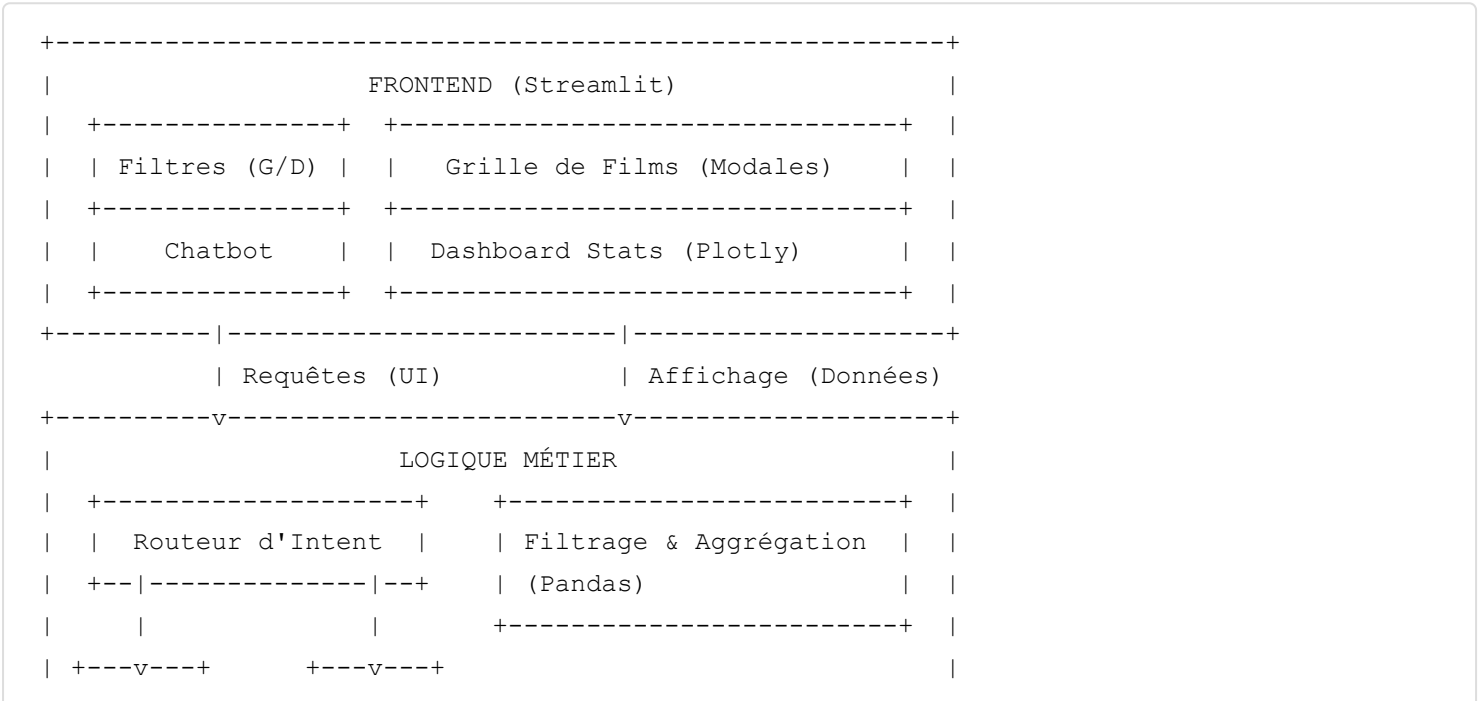
Note de Synthèse : Plateforme de Visualisation "MovieDB Explorer" (Hi! PARIS)

1. Méthodologie

Notre démarche s'est articulée autour de la conception d'une plateforme robuste, interactive et esthétique, mettant l'expérience utilisateur au centre tout en exploitant des technologies avancées d'analyse de données et d'Intelligence Artificielle.

- Exploration et Prétraitement des données :** La base `movies.csv` a été nettoyée avec la bibliothèque `pandas`. Les valeurs manquantes (titres, dates, affiches) ont été sécurisées par des filtres ou des valeurs par défaut. Les dates de sortie ont été formatées en objets `datetime` pour permettre le filtrage chronologique.
- Architecture Frontend (Choix Technologique) :** Nous avons choisi le framework **Streamlit** en Python au détriment d'une architecture plus lourde de type React.js (Frontend) + FastAPI (Backend). Ce choix est justifié par le temps limité imparti pour réaliser ce challenge (2 à 3 heures). Streamlit permet un prototypage ultra-rapide et une intégration directe avec l'écosystème IA/Data (Pandas, LangChain, HuggingFace), évitant la gestion complexe des API, tout en offrant un résultat très professionnel grâce à une refonte complète de l'interface (CSS personnalisé aux couleurs de **Hi! PARIS**).
- Intégration de l'Intelligence Artificielle :** La plateforme intègre un moteur de requêtes hybride utilisant **LangChain**, combinant le traitement du langage naturel avec l'exécution de requêtes sémantiques. Le pipeline intelligent détecte l'intention de l'utilisateur (Salutation, Recherche par RAG, Requête analytique Pandas) et aiguille la requête vers le composant approprié.

Schéma d'Architecture de l'Application



	Agent		Agent	
	Pandas		RAG	
	+--- ---+		+--- ---+	
+	----- -----	+	----- -----	+
	Code Python		Requête Sémantique	
+	-----v-----v-----	+	-----v-----v-----	+
	BACKEND & IA (Services)			
	+-----+ +-----+		+-----+ +-----+	
	LLM HuggingFace API		FAISS Vector DB	
	(Llama-3-8B-Instruct)		(SentenceTransformers)	
	+-----+ +-----+		+-----+ +-----+	
	+-----+			
	movies.csv (Données Brutes)			
	+-----+		+-----+	
+	----- -----	+	----- -----	+

2. Fonctionnalités Principales

A. Exploration Visuelle Multimodale et Filtrage Dynamique

La plateforme présente les données de manière visuelle sous forme de **cartes interactives** contenant du texte (titre, dates, notes) et des images (affiches). Des filtres latéraux multicritères (recherche de titre, sélection de genres, année de sortie, et sliders de notes) permettent d'affiner dynamiquement le jeu de données. Un clic sur le bouton "Voir la fiche" ouvre une fenêtre contextuelle détaillée pour une lecture ergonomique.

Figure 1 : Interface Globale et Multimodalité

(Recommandation de capture : Prenez une capture en plein écran de l'onglet "Exploration Visuelle" avec la barre latérale des filtres à gauche, les cartes de films au centre, et ouvrez une modale "Voir la fiche" par-dessus. Cela illustrera la richesse visuelle, les couleurs Hi! PARIS, et la multimodalité texte/image).

B. Assistant Cinéma IA Hybride (RAG + Agent Pandas)

Un agent conversationnel flottant a été développé pour interagir en langage naturel avec la base de données. Il possède deux moteurs :

- Recherche Sémantique (RAG) :** Utilisation de `HuggingFaceEmbeddings` et `FAISS` pour la recherche par similarité (ex: "Trouve un film parlant de voyage dans le temps"). L'IA lit les synopsis pertinents et argumente son choix.
- Exécution dynamique de requêtes :** Si la question est analytique ("Quels sont les 5 films d'action avec les meilleures notes ?"), le LLM traduit la requête en code Python `pandas`, l'exécute localement et injecte un véritable tableau de données dans la discussion. Une liste déroulante permet même de consulter la fiche détaillée des films trouvés depuis le Chat.

Figure 2 : Assistant IA et Interactivité

(Recommandation de capture : Prenez une capture centrée sur le Popover du Chatbot

ouvert (en bas à droite). Posez-lui une question comme "Les films d'action avec une note > 8" pour qu'il affiche un tableau de données, et montrez le menu déroulant "Inspecter un film". Cela prouve l'intégration de Pandas dans le chat).

C. Analyse Statistique Graphique

Le deuxième onglet propose une visualisation macroscopique via **Plotly**. Les utilisateurs peuvent analyser la distribution des notes moyennes ou étudier la popularité des genres cinématographiques au fil des décennies, rendant l'outil pertinent pour des analyses exploratoires de haut niveau.

3. Limites et Améliorations Possibles

Limites Actuelles

- **Maturité du Chatbot IA** : Par manque de temps de développement dans le cadre du challenge, l'Agent conversationnel ne fonctionne pas encore de manière optimale sur les requêtes très complexes et manque de garde-fous sur les cas d'erreur.
- **Portabilité de la Base Vectorielle** : La création et l'utilisation d'un index vectoriel `FAISS` en local ne sont pas adaptées pour partager ou déployer facilement le projet (fichiers lourds, temps de compilation, dépendance forte à l'environnement).
- **Hallucinations du Modèle (Génération de code)** : Bien que l'agent Pandas soit guidé par un prompt strict, les modèles Open Source limités (taille < 8B) peuvent parfois générer un code syntaxiquement incorrect, provoquant l'échec de la requête.

Améliorations Envisageables

- **Base de Données Vectorielle Dédiée** : Migrer les vecteurs FAISS vers une infrastructure dédiée (Pinecone, Milvus ou Qdrant) pour assurer un requêtage instantané asynchrone sur de très larges bases de données.
- **Système de Recommandation Algorithmique** : Intégrer un filtrage collaboratif classique en complément de la recherche sémantique, permettant de recommander des films similaires basés sur des patterns de notes d'utilisateurs.

4. Usage des Outils d'IA dans le Challenge

L'utilisation de l'intelligence artificielle (agent développeur) a été systématique pour maximiser l'efficacité tout au long de ce projet :

- **Tâches déléguées à l'IA (LLM Assistant)** :
 - **Prototypage Rapide** : Génération du code initial pour charger et nettoyer le fichier `movies.csv`.
 - **Architecture de l'Interface** : Mise en place des structures en colonnes Streamlit et injection des règles CSS avancées (modales, popovers ancrés, design system aux couleurs Hi!

PARIS, animations CSS).

- **Conception Logique du Moteur IA** : Rédaction de la logique métier hybride (RAG via FAISS + Exécution sécurisée de scripts Pandas dans Streamlit).

- **Vérifications et corrections humaines** :

- **Contrôle Qualité** : Une relecture critique a été opérée sur la boucle d'affichage de la grille de films pour corriger des erreurs d'indexation (gestion d'index `pandas` provoquant des trous dans l'affichage).
- **Recadrage Technique** : Validation du fonctionnement de l'API HuggingFace et adaptation du code pour contourner les blocages liés au proxy SSL, ainsi que des itérations poussées de *prompt engineering* pour s'assurer que le Chatbot renvoie bien des objets Python exécutables, et non du format Markdown inutilisable.
- **Ergonomie** : L'ajustement millimétré des balises HTML/CSS a nécessité des itérations pour rendre l'interface lisible, responsive et accessible (ex: résolution des conflits de thèmes sombres/clairs).

5. Guide de Lancement et Configuration IA

Lancement depuis Git (Local)

Pour reproduire ce projet en local, il vous suffit de cloner le dépôt Github et de lancer l'application Streamlit. Assurez-vous d'avoir Python 3.9+ installé.

```
# 1. Cloner le répertoire
git clone <URL_DU_REPO>
cd <NOM_DU_REPO>

# 2. Installer les dépendances
pip install -r requirements.txt

# 3. Lancer l'application
python -m streamlit run app.py
```

(L'application s'ouvrira automatiquement dans votre navigateur par défaut à l'adresse `http://localhost:8501`).

Configuration de l'Intelligence Artificielle (HuggingFace)

Pour la partie Chatbot et Recherche Sémantique (RAG), nous avons fait le choix délibéré d'utiliser des modèles **Open Source** de l'écosystème HuggingFace plutôt que des API fermées (type OpenAI GPT-4).

- **Justification de l'Open Source** : L'utilisation de modèles libres tels que `Meta-Llama-3-8B-Instruct` (pour l'Agent Conversationnel) et `sentence-transformers` (pour l'indexation FAISS) permet une meilleure maîtrise des coûts, garantit la souveraineté des données analysées, et

s'aligne parfaitement avec les principes de transparence et de partage du milieu académique (Hi! PARIS).

- **Token API HuggingFace** : L'application utilise l'Inference API gratuite de HuggingFace. Pour faciliter l'évaluation du projet par n'importe quel utilisateur, **un Token personnel est fourni par défaut** directement dans l'interface et le code : `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`
- **Conseils d'utilisation** : Lors du lancement de l'application, ce Token est déjà pré-rempli dans la barre latérale gauche (section "*Configuration IA (LLM)*"). Vous n'avez **aucune configuration manuelle à faire** pour tester le Chatbot immédiatement.
- **En cas d'épuisement du quota (Comment générer un nouveau Token)** : Si de trop nombreuses requêtes sont effectuées, l'API de HuggingFace bloquera temporairement les appels gratuits. L'utilisateur peut alors générer son propre Token gratuit en 1 minute en suivant ces étapes :
 1. Rendez-vous sur huggingface.co et créez un compte gratuit (bouton *Sign Up*).
 2. Allez dans les paramètres de votre profil, puis dans la rubrique **Access Tokens** (ou directement sur le lien huggingface.co/settings/tokens).
 3. Cliquez sur le bouton **Create new token**.
 4. Donnez un nom au token, choisissez le type "Fine-grained" ou simplement les droits de lecture ("Read"), puis générez-le.
 5. Copiez la clé secrète générée (qui commence par `hf_...`).
 6. Retournez sur l'application Streamlit et **collez cette clé** dans la case "*HuggingFace API Key (Token)*" située dans la barre latérale. Le Chatbot sera instantanément de nouveau opérationnel avec votre quota neuf.